

Lista problemów na Noc z Julią

1. (ogólne) animacja modelu Isinga (są różne algorytmy do tego), przykładowa realizacja w mowie węża: <https://github.com/falcotinnunculus/MC-simulations-course/tree/master/106>
2. (ogólne) jakdojade (wczytywanie listy linii tramwajowych, konstrukcja grafu jako tabeli sąsiedztwa, algorytm Dijkstry, przypisywanie wynikowi linii) (zestaw 6 u Przygody)
3. (ogólne) prosta gra typu piłka w polu grawitacyjnym (nie mam pojęcia czy istnieje pygame.jl) *Answer: Niewiem, ale istnieje GameZero.jl* (zestaw 4 u Przygody)
4. (ogólne) Napisz symulację modelu epidemiologicznego na sieci (najprostszej: np. krata z periodycznymi warunkami brzegowymi)
5. (ogólne) Napiszcie spektrusia w Julii. <https://1pf.if.uj.edu.pl/materialy-do-cwiczen-programy-instrukcje-materialy-dodatkowe> albo https://www.youtube.com/watch?v=DV2W_Yn8rhk
6. (ogólne) lokalizator wolnych sal na wydziale (Great Obfuscated Localizator of Available Classrooms)
7. (ogólne) przeglądnąć listę projektów zaliczeniowych z mowy węża <https://ufkapano.github.io/algorytmy/tematy.html> i wybrać coś ciekawego (np. grę w życie)
8. (ODE) Porównaj kilka najprostszych algorytmów całkowania numerycznego. Euler, Runge-Kutta, inexplicit Euler. Pierwszych dwóch bym nie implementował, tylko nauczył się panować nad *DifferentialEquations.jl*
9. (ODE) Wyznacz funkcję Lane-Emdena do pierwszego miejsca zerowego (i jego położenie). https://en.wikipedia.org/wiki/Lane%E2%80%93Emden_equation W tym zadaniu chodzi o to, że *DifferentialEquations.jl* wspiera coś co się u nich nazywa callback https://docs.sciml.ai/DiffEqDocs/stable/features/callback_functions/. Możesz też zrobić to inaczej (w ramach nauki nowej biblioteki), czyli rozwiązać dla dużego przedziału $t \in [0, T]$ a potem użyć biblioteki do szukania miejsc zerowych *Roots.jl*.

10. (ODE) Zbadaj model Kuramoto synchronizacji owadów (cool zabawka). Można go rozwiązywać znacznie optymalniej jeśli wprowadzi się odpowiednie parametry porządku.
11. (SDE) Zbadaj model Kuramoto synchronizacji owadów ale dodaj szum, żeby mieć SDE zamiast ODE. (Myślę, że ciekawe tylko dla ludzi którzy wiedzą co to SDE)
12. (PDE) Zapoznaj się z metodami rozwiązywania PDE (np. przepływ ciepła z dyskretyzacją przestrzeni). Są do tego biblioteki <https://docs.sciml.ai/MethodOfLines/stable/>, są obliczenia półsymboliczne. Ja tego nigdy nie robiłem i ogólnie to jest bardzo trudna tematyka bo PDE potrafią być strasznie nieregularne.
13. (ODE) przeglądając listę projektów zaliczeniowych z mowy Szatana <https://th.if.uj.edu.pl/~odrzywolek/homepage/students/narzedzia/> i wybrać te które są robialne w Julii (np. solar sail, wahadło, choinka, koralik, automatyczna 1PF (ale to nie ODE)).
Alternatywne sformułowanie zadania: wybierz swoje ulubione zadanie z Mechaniki Klasycznej i przepisz do Julii
14. (Optymalizacja) Znajdź ekstremum funkcji kwadratowej i jej miejsce zerowe używając bibliotek do tego *Roots.jl*, *Optimization.jl*. Potem można zastosować to samo do jakichś ciekawszych problemów.
15. (DataAnalysis) Tu są dane z doświadczenia na 2pf i sprawozdanie. Trzeba powtórzyć analizę, ale korzystając z Julii. Nauczy to pakietów *DataFrames.jl*, *GLM.jl*, *LsqFit.jl* i może czegoś jeszcze.
16. (Statystyka) Tu są dane, proponuję je zwizualizować i np. dofitować model liniowy.
17. (DataAnalysis) Znajdź jakieś ciekawe dane i pokaż ile możesz z nich wyciągnąć.